

02 Algorithmik Asymptotische Analyse

Daniel Gaida

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

17.07.2021

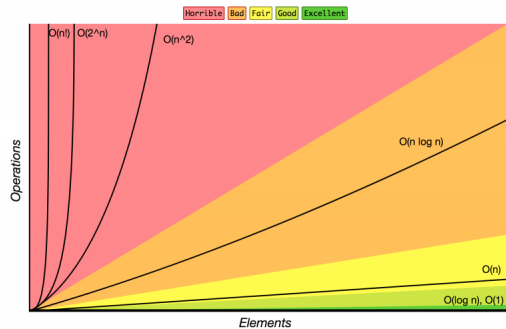
Lernraum I: Abstrakte Datentypen, Datenstrukturen und Asymptotische Notation

- Abstrakte Datentypen
- Liste, Stapel, Warteschlange, Tabelle (Dictionary)
- Datenstrukturen
- Array, Verkettete Liste
- Implementierungen der Abstrakten Datentypen mit den Datenstrukturen ...
- Array
- Verkettete Listen
- Analyse von Laufzeit und Speicherbedarf der Implementierungen
- O-Notation
- Laufzeitanalyse von Algorithmen durch Asymptotische Notation
- O-Notation, Ω -Notation, Θ -Notation
- Fallanalyse: Worst Case, Best Case Analyse
- Laufzeitanalyse von Rekursiven Algorithmen durch Asymptotische Notation

Klassifikation der Wachstumsordnung

Wachstumsordnung: Eine Kategorie der Algorithmeneffizienz, die auf dem Verhältnis des Algorithmus zur Eingabegröße n basiert.

Description	O-Notation	Runtime if you double n	Example Algorithm
constant	$O(1)$	unchanged	Accessing an index of an array
logarithmic	$O(\log_2 n)$	increases slightly	Binary search
linear	$O(n)$	doubles	Looping over an array
log-linear	$O(n \log_2 n)$	slightly more than doubles	Merge sort algorithm
quadratic	$O(n^2)$	quadruples	Nested loops!
exponential	$O(2^n)$	multiplies drastically	Fibonacci with recursion



Code in O-Notation umwandeln

CODE

```
1 for (i = 0; i < n; i++)  
  {  
2   a[i] = 1;  
3   b[i] = 2;  
4  }
```

- 5
- Bisher: “ $O(1)$ bedeutet keine Schleifen, $O(n)$ ist eine Schleife, $O(n^2)$ ist verschachtelte Schleife”
 - Das ist weiterhin nützlich!
 - In diesem Modul gehen wir mehr ins Detail: Damit wir auch komplizierteren Code in die O-Notation umwandeln können



O-Notation

$O(n)$

Analyse von Algorithmen

1

2

CODE

Code Modellierung

LAUFZEIT FUNK-
TION

Wachstums-ordnung

3Asymptotische Ana-
lyse

$f(n) = 2n$

$O(n)$

```
1 for (i = 0; i < n;
   i++) {
2   a[i] = 1;
3   b[i] = 2;
4 }
5
```

- Analyse von Algorithmen: Der Gesamtprozess der Zuordnung von Code zu einer Wachstumsordnung, bestehend aus:
- Code Modellierung: Code $\rightarrow$ Laufzeitfunktion, die die Laufzeit des Codes beschreibt
- Asymptotische Notation: Funktion $\rightarrow$ Wachstumsordnung, die das asymptotische Verhalten beschreibt

Code Modellierung

1

CODE

LAUFZEIT

Code Modellierung

Code Modellierung - der Prozess der mathematischen Darstellung, wie viele Operationen ein Stück Code im Verhältnis zur Eingabegröße n ausführt. von Code in eine Laufzeitfunktion, die seine Laufzeit darstellt

Code Modellierung Beispiel 2

```

1 public int method2(int n) {
2   int sum = 0;
3   int i = 0;
4   while (i < n) {
5     int j = 0;
6     while (j < n) {
7       if (j % 2 == 0) {
8         // do nothing
9       }
10      sum = sum + (i * 3) + j;
11      j = j + 1;
12    }
13    i = i + 1;
14  } return sum;
15 }
16

```

+2

+4

+2

+2

+1

Innere Schleife
läuft n-mal

+9

Äußere Schleife
läuft n-mal

$9n + 4$

$*n*n$

O-, Ω - und Θ -Notation Übung

Welche der folgenden Funktionen ist in $O(n^2)$? $\Omega(n^2)$?

a.

$$f(n) = O(n^2)$$

b.

$$f(n) = O(n^2)$$

c.

$$f(n) = O(n^2)$$

d.

$$f(n) = O(n^2)$$

$$f(n) = \Omega(n^2)$$

$$f(n) = \Theta(n^2)$$

e.

$$f(n) = \Omega(n^2)$$

O-Notation

$f(n) \in O(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $0 \leq f(n) \leq c \cdot g(n)$ für alle $n \geq n_0$

Ω -Notation

$f(n) \in \Omega(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $f(n) \geq c \cdot g(n)$ für alle $n \geq n_0$

Θ -Notation

$f(n) \in \Theta(g(n))$ falls $f(n) \in O(g(n))$ und $f(n) \in \Omega(g(n))$.
 $f(n) \in \Theta(g(n))$ falls positive Konstanten c_1, c_2 und n_0 existieren, sodass
 $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ für alle $n \geq n_0$

Analyse von Algorithmen

			2	
	1			$O(n)$
CODE		LAUFZEIT		
	Code Modellierung	FUNKTION	Asymptotische Analyse	O-Notation Ω -Notation Θ -Notation $\Theta(n)$
		$f(n) = 2n$		
1 for (i = 0; i < n; i++) { 2 a[i] = 1; 3 b[i] = 2; 4 } 5				

Schauen wir uns dieses Werkzeug nun genauer an. Wie genau kommen wir zu dieser Funktion?

Wir haben gerade dieses Werkzeug fertiggestellt, um eine Funktion anhand einiger nützlicher Schranken zu charakterisieren!

Temporary page!

$\text{\LaTeX}$ was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to the document. If you rerun the document (without altering it) this surplus page will go away, because $\text{\LaTeX}$ now knows how many pages to expect for this document.